MANIPAL UNIVERSITY JAIPUR

University under Section 2(f) of the UGC Act

NAME	Awad Ali
ROLL NUMBER	2414509633
SESSION	SEP 2025
PROGRAM	BACHELOR OF COMPUTER APPLICATIONS
SEMESTER	II
COURSE CODE	DCA1207
COURSE NAME	DATA STRUCTURES
CREDIT	4
NUMBER OF ASSIGNMENT AND MARKS	02 30 Marks each

Ans1.

Arrays are one of the most basic ways that programmers store multiple pieces of data of the same type in order. Arrays are mostly divided into two groups based on how they are built and used: one-dimensional arrays and multi-dimensional arrays.

The simplest type of an array is a one-dimensional array. It keeps things in a straight line and lets you get to them with just one index. A one-dimensional array is, for example, an array that holds students' grades or a list of numbers. Because it's simple and easy to use, a lot of people use it.

A multi-dimensional array has more than one dimension, and it is usually shown as rows and columns. A two-dimensional array is the most common type. It looks like a table or a matrix. It is used to keep things like matrices, pictures, or tables of data. It is possible to have arrays with more than two dimensions, like three-dimensional arrays, but they are not used as often.

How to Remove an Element from the Middle of an Array

Because arrays store elements in contiguous memory locations, deleting an element from the middle of an array is not a direct operation. When you take something out of an array, the other elements

have to move to keep the array's structure.

Step 1: Put in the Array
First, take the elements of the array and save them in an array. Also, remember the size of the array and where the element you want to delete is (index).

Step 2: Check the Position
Make sure that the position you were given is correct. If the position is less than 0 or greater than or equal to the size of the array, you can't delete it, and an error message should show up.

Step 3: Move the Parts
From the position of deletion, move each element one spot to the left. This means that the item at index i+1 is copied to index i. This process goes on until the second-to-last element of the array.

Step 4: Make the array smaller
After moving the items, the logical size of the array should be decreased by one because one item has been removed.

Step 5: Show the New Array
Finally, print the array elements up to the new size to show what happened after the deletion.

This algorithm makes sure that the array stays in the right order after you delete something. Even though it takes a long time to delete an item from the middle of an array because it has to be moved, this is a common task in programming, such as managing data and processing lists.

Ans2.
An array is a basic data structure in programming that keeps items of the same type in a memory space that is next to each other. A lot of the time, programmers need to find a certain value in an array and change it to a different one. This operation is common in programs that need to update records, fix data, or change values based on certain conditions. A clear and logical algorithm is needed to do this job well.

An algorithm to find and change a number in an array

Step 1: Begin
Define the array and set up the necessary variables to start the algorithm. The size of the array, the element to be searched (called the target), and the new value that will take the place of the target are all examples of these.

Step 2: Enter the elements of the array
You can either let the user choose the elements of the array or set them up in the program. Also, type in the number you want to find and the number that will replace it.

Step 3: Start a loop
Make a loop that starts at the first index of the array and goes to the last index. This makes sure that every part of the array is checked.

Step 4: Look at each part and compare it.
Within the loop, check to see if the current array element is equal to the target value. This comparison tells you if you've found the right element.

Step 5: Swap out the Element
If you find a match, change the current element to the new value. The algorithm may only replace the first occurrence of the target value or all occurrences of it, depending on what is needed.

Step 6: Go on with the loop or leave it
If only one replacement is needed, the loop can end right after it is done. If you can make more than one replacement, the loop goes on until all the elements have been checked.

Step 7: Show the New Array
After you finish the search and replace operation, show the updated array to show what has changed.

Step 8: Stop. Finish the algorithm.

How the Algorithm Works

This algorithm uses a linear search method, which means that it looks at each element of the array one at a time. Linear search is easy to use and works well, especially on small or unsorted arrays. The time complexity of this algorithm is O(n), where n is the number of elements in the array, because it might have to check each one once.

This operation is memory-efficient because replacing an element doesn't require moving other elements or changing the size of the array. The algorithm is flexible because you can easily change it to replace more than one occurrence, count how many replacements were made, or stop after the first match.

In conclusion, if you do it the right way, finding and replacing a specific number in an array is easy. Programmers can make sure that array data is updated correctly by following this algorithm, which keeps their code simple and clear.

Ans3.
A stack is a linear data structure that works on the LIFO (Last In, First Out) principle. This means that the last item added to the stack is the first one to be taken out. A stack can only do things at one end, which is called the top of the stack. Push (adding an element) and pop (removing an element) are the two main things you can do with a stack. Peek is another common operation that lets you see the top element without taking it out.

How a Stack Data Structure Works

The push operation adds an element to the stack by putting it on top of the other elements. The top pointer then moves up. Stack overflow happens when you try to push something onto a stack that is

already full.

In the same way, the pop operation takes the top item off the stack. The top pointer moves down after the item is taken away. Stack underflow happens when you try to pop an empty stack.

You can use arrays or linked lists to make stacks. The size of the stack is fixed in an array implementation, but it can change in a linked list implementation. Stacks are easy to use and manage because they have simple operations and strict access rules.

How Stack Can Be Used in the Real World

Because of their controlled access structure, stacks are used a lot in computer science and in real life.

Function calls and recursion are two common uses. A call stack keeps track of the details of a function when it is called. When the function is done running, it is taken off the stack. This helps programs handle many function calls at once.

Stacks are also used to evaluate expressions and parse syntax, like turning infix expressions into postfix ones or evaluating postfix expressions in calculators and compilers.

Undo and redo are also very useful in software programs like text editors. When you do something, it goes on the stack, which lets you undo or redo actions in order.

Stacks are also used to help people navigate their browser history. They keep track of the pages that have been visited so that users can easily go back and forth.

Stacks are also very important for managing memory, backtracking algorithms, and checking for balanced parentheses in programming languages.

In the end, the stack data structure is both simple and powerful. Because it works in a last-in, first-out (LIFO) way, it is very useful for efficiently managing tasks, memory, and processes in both software systems and real life.

Ans4.
A linked list is a linear data structure that stores a group of elements called nodes. Each node has two parts: data and a link (or pointer) to the next node in the list. Linked lists don't store elements in memory locations that are next to each other like arrays do. Instead, pointers are used to connect nodes dynamically, which makes it easy to add and remove nodes.

Different kinds of linked lists

One Linked List
In a singly linked list, each node has data and a pointer to the next node. You can only go one way when you traverse (from first to last). It is easy to use and takes up less memory than other linked lists.

Linked List with Two Links
A node in a doubly linked list has three parts: data, a pointer to the next node, and a pointer to the node before it. This lets you move both forward and backward. It needs more memory, but it gives you more options.

Linked List in a Circle
In a circular linked list, the last node doesn't point to NULL; instead, it points back to the first node. This type can be circular once or twice. It is useful for tasks that need to be done over and over, like scheduling.

Doubly Linked List in a Circle
This is a mix of doubly and circular linked lists, where nodes are linked in both directions and make a loop.

Advantages of Linked Lists Compared to Arrays

One of the best things about linked lists is that their size can change. You have to set the size of an array ahead of time, which can waste memory or cause an overflow. Linked lists can grow or shrink while the program is running because memory is allocated dynamically.

Another big plus is that it's easy to add and remove items. When you add or remove an element from an array, you have to move other elements around, which takes time. Changing pointers makes it easy to add and remove items from linked lists, especially in the middle of the list.

Linked lists also use memory more efficiently. Arrays need memory that is next to each other, which may not always be available. Linked lists use memory locations that aren't next to each other, which makes them more flexible when it comes to memory allocation.

Linked lists are also helpful for making other data structures like hash tables, stacks, queues, and graphs. They work best when you need to make changes often.

But linked lists have some problems, like using more memory for pointers and being slower to access because you can't get to elements directly by index. Even so, linked lists are better than arrays in situations where being able to change data easily and quickly is more important than being able to access it quickly at random.

Ans5.
A doubly circular queue is a linear data structure that has the same properties as both a circular queue and a doubly linked list. Each node in this structure has three parts: a data field, a pointer to the node that came before it, and a pointer to the node that comes after it. When something is circular, it means that the last node is connected to the first node, and the first node is also connected to the last node, making a closed loop. The doubly circular queue is more flexible and efficient in some cases because it lets you move both forward and backward, unlike a simple circular queue.

There is no NULL pointer in a doubly circular queue. The first node's prev pointer points to the last node, and the last node's next pointer points to the first node. This structure is very helpful when you need to add and remove items often from both ends and when you need to move through it in both directions. People often use it for things like scheduling tasks, navigation systems (undoing

and redoing actions), multimedia playlists, and systems for managing resources.

How a Doubly Circular Queue Works

The queue works on the "first in, first out" (FIFO) principle. You add elements to the back and take them out of the front. Because memory is circular, it is used well because the queue can use spaces that have been freed up again. The doubly linked structure makes it easy to move in both directions without changing the data.

How to Show the Items in a Doubly Circular Queue

To display the queue, we start at the front node and go all the way to the back node and back again. To avoid infinite looping, you need to be careful.

Algorithm: Show_Doubly_Circular_Queue

- **Start**
- Check if the queue is empty
  - If `front == NULL`, then
    - Print “Queue is empty”
    - Go to Step 7
- Set a temporary pointer `temp = front`
- Repeat the following steps:
  - Print `temp->data`
  - Move `temp = temp->next`
- Continue Step 4 **until `temp == front` again**
- Stop traversal
- **End**

How the Algorithm Works

To avoid mistakes, the algorithm first checks to see if the queue is empty.

If the queue isn't empty, a temporary pointer is set to the front of the queue.

The loop prints the information for each node and then goes on to the next node.

The traversal stops when the pointer gets back to the front node because the queue is circular.

This makes sure that each element is shown only once.

Benefits of a doubly circular queue

Because of its circular shape, it uses memory well.

You can go both ways through it.

Faster to add and remove than arrays

No need to move things around

In conclusion, a doubly circular queue is a very useful data structure that makes queue-based apps more flexible, efficient, and faster.

Ans6.
The Merge Sort Algorithm and Its Way of Dividing and Conquering

Merge Sort is a well-known and effective sorting method that uses the divide-and-conquer method. It sorts items in an array or list by breaking the data into smaller pieces, sorting them, and then putting them back together in a sorted way. Merge Sort is a good choice for sorting large datasets because it works consistently and has a stable time complexity.

The Divide-and-Conquer Method

The divide-and-conquer method has three main steps:

Divide: The array that was given is split into two equal (or almost equal) parts. This process keeps going until each subarray only has one element. One element is already thought to be sorted.

Conquer: The algorithm starts sorting the array once it has been split into the smallest possible subarrays. No comparison is needed at this point because each subarray only has one element.

Combine (Merge): The sorted subarrays are then combined one at a time to make bigger sorted subarrays. When merging, the elements of two subarrays are compared, and the smaller one is put first in a temporary array. This process goes on until all the elements are combined in the right order.

This methodical way of doing things makes sure that the final output is a completely sorted array.

Algorithm for Merge Sort

MergeSort(arr, left, right):

If left is less than right, then

Find the middle index by adding left and right and dividing by 2.

Call MergeSort(arr, left, and mid)

Use MergeSort(arr, mid + 1, right)

Use Merge(arr, left, mid, right) to combine the two sorted halves.

Combine(arr, left, mid, right):

Make two temporary arrays to hold the left and right subarrays.

Put data into temporary arrays

Look at the things in both arrays and see how they are different.

Put the smaller item back into the original array.

If there are any, copy the rest of the elements.

How It Works

At first, the array is split up many times until each subarray only has one element. After that, merging starts at the bottom level. At each merge step, two sorted subarrays are put together to make a new sorted array. This process goes on until all of the items in the array are combined into a single sorted list.

For instance, if we sort the array {38, 27, 43, 3, 9, 82, 10}, it is split up until there is only one element left, and then it is merged back together in sorted order.

The benefits of Merge Sort

In all cases (best, average, and worst), the time complexity is O(n log n).

Stable sorting algorithm (keeps the order of equal elements the same)

Good for linked lists and big datasets

In conclusion

Merge Sort is a sorting algorithm that works well and is based on the "divide and conquer" method. It makes sure that performance and accuracy are always the same by breaking the problem down into smaller parts and combining the sorted results. This is why it is so widely used in computer science and in the real world.